# Trailing Stop, Break-Even and Position-Sizing Math for Leveraged Trading Bots

## Introduction

Trading bots that operate in leveraged futures markets need carefully designed logic for sizing positions, calculating profits and losses (P&L), managing risk and closing trades. Leveraged trading amplifies both gains and losses: a 0.2 % move in the underlying price can correspond to a 2 % change in account equity on a 10× leveraged position. This manual consolidates the mathematics used in version 3.4 of the user's trading bot together with improvements proposed for version 3.5. It extends the original reference by including trading fee adjustments, liquidation price formulas, slippage estimators, ROI-based stop logic, volatility-dependent leverage selection and practical safety considerations for interacting with exchange APIs.

## 1 Position sizing with leverage

Leveraged futures exchanges require traders to post margin (collateral) while allowing them to control a larger notional position. The following formulas compute the size of each trade:

- **Margin used**: allocate a percentage of the account for each position. Let $B$ be the account balance in USDT and $p$ be the position size percentage; the margin used is

$$\text{marginUsed} = B \times \frac{p}{100}.$$

For example, risking 0.5 % of a $10,000 account uses $50 as margin.

- **Notional exposure**: apply leverage $L$ to the margin to obtain the total position value:

$$\text{positionValue}_{USD} = \text{marginUsed} \times L.$$

Using 10× leverage on $50 produces $500 of notional exposure.

- **Contract size**: compute the number of contracts (lots) by dividing the position value by the entry price $P_0$ and the contract multiplier $m$ (1 for USDT-quoted contracts), then rounding down to the nearest lot:

$$\text{size} = \left\lfloor \frac{\text{positionValue}_{USD}}{P_0 \times m} \right\rfloor.$$

- **Actual exposure and margin**: after rounding, the actual notional exposure and margin are

$$\text{actualPositionValue}_{USD} = \text{size} \times P_0 \times m,$$

$$\text{actualMarginUsed} = \frac{\text{actualPositionValue}_{USD}}{L}.$$

These formulas mirror the position-sizing reference used in version 3.4 of the bot and align with standard futures trading practice【109569485076299†L23-L47】.

## 2 P&L calculation and leveraged percentages

For each tick the bot calculates both the unrealised dollar P&L and the leveraged percentage return:

- **Price difference**: for a long position the price difference is $\Delta P = P - P_0$, while for a short position $\Delta P = P_0 - P$.
- **Unrealised P&L** (USDT): multiply the price difference by the position size and contract multiplier:

$$\text{unrealizedPnl} = \Delta P \times \text{size} \times m.$$

- **Leveraged P&L percentage**: divide the unrealised P&L by the margin used and convert to a percentage:

$$\text{leveragedPnlPercent} = \frac{\text{unrealizedPnl}}{\text{marginUsed}} \times 100.$$

Using the leveraged percentage rather than a raw price percentage avoids the error of earlier versions which moved stops after a tiny price change. For example, a 0.2 % price move on a 10× position yields a 2 % return on the margin【109569485076299†L61-L78】.

## 3 Stop-loss and take-profit logic

### 3.1 Price-based stops (v3.4)

Version 3.4 of the bot defines the initial stop-loss (SL) and take-profit (TP) as fixed price percentages relative to the entry price. If $s$ is the stop percentage and $t$ is the take-profit percentage:

- **Long positions**: $\text{SL} = P_0 \times \left(1 - \frac{s}{100}\right)$, $\text{TP} = P_0 \times \left(1 + \frac{t}{100}\right)$.
- **Short positions**: $\text{SL} = P_0 \times \left(1 + \frac{s}{100}\right)$, $\text{TP} = P_0 \times \left(1 - \frac{t}{100}\right)$.

These values are used to place conditional stop and limit orders via the exchange API.

### 3.2 ROI-based stops (v3.4.1 and later)

The standard price-based approach scales poorly with leverage. A 1 % stop in price translates to a 10 % loss on margin when using 10× leverage. Version 3.4.1 introduces **inverse leverage scaling** so that stop and take-profit distances are defined by the desired return on investment (ROI) rather than raw price:

Let $R_{\text{risk}}$ and $R_{\text{reward}}$ be the target ROI percentages for the stop-loss and take-profit, and let $L$ be the leverage. The required price movements for a long position are

$$\text{SL price} = P_0 \times \left(1 - \frac{R_{\text{risk}}}{L}\right), \text{TP price} = P_0 \times \left(1 + \frac{R_{\text{reward}}}{L}\right).$$

For a short position the signs of the price adjustments are reversed. In the implementation, the bot calculates

```
const slPercent = (CONFIG.TRADING.INITIAL_SL_PERCENT / leverage) / 100;
const tpPercent = (CONFIG.TRADING.INITIAL_TP_PERCENT / leverage) / 100;
```

which effectively divides the ROI targets by the leverage. This ensures that a 2 % take-profit means a 2 % gain on account equity regardless of the leverage used.

## 4 Break-even logic

Moving the stop to the entry price when a trade shows a small profit removes downside risk. The bot uses a tiny leveraged P&L threshold (0.001 % by default) to trigger break-even. When leveragedPnlPercent exceeds this threshold, the current stop is set equal to the entry price and the break-even flag is recorded. Subsequent trailing stops are measured from this new baseline 【109569485076299†L86-L103】.

## 5 Trailing stop algorithm

Trailing stops follow the price at a defined distance to lock in profits as the market moves in the trader's favour. Version 3.4 implements a **staircase** algorithm parameterised by two constants:

- **TRAILING_STEP_PERCENT** (default 0.15 % ROI): the minimum increment of leveraged P&L required before moving the stop again.
- **TRAILING_MOVE_PERCENT** (default 0.05 % in price): the amount by which the stop moves per step.

Each time the profit percentage increases beyond the last trailed level, the bot computes

$$\text{stepsSinceLastTrail} = \lfloor \frac{\text{leveragedPnlPercent} - \text{lastTrailingLevel}}{\text{TRAILING\_STEP\_PERCENT}} \rfloor,$$

and, if this value is positive, moves the stop by $\Delta_{SL} = \text{stepsSinceLastTrail} \times \text{TRAILING\_MOVE\_PERCENT}$ of price. For long positions the stop is multiplied by $(1 + \Delta_{SL}/100)$ and for shorts by $(1 - \Delta_{SL}/100)$ 【109569485076299†L137-L159】. The stop never moves backwards, ensuring that locked-in profits are preserved.

## 6 Trading fees and net P&L calculation

Unrealised P&L does not account for transaction fees. In leveraged futures markets, exchanges charge maker or taker fees on the **notional value** of the trade (the margin multiplied by the leverage). As a result, fees have a larger impact on equity than they would in spot trading. For example, Bitget's support centre explains that the opening fee for a USDT-denominated futures trade is computed as the entry price multiplied by the fee rate 【719692000825004†L82-L91】, and the closing fee is calculated using the exit price 【719692000825004†L90-L92】. Funding

fees are calculated as the position value multiplied by the funding rate 【719692000825004†L94-L99】.

To determine the net realised P&L when closing a position, subtract all fees from the gross profit:

[ = (P_{} - P_0) m - - - . ]

Assuming the trader pays the taker fee $F$ on both entry and exit, the total fee equals $2F \times \text{positionValue}_{USD}$. The leveraged ROI required to break even after fees is therefore

[ = 2 F L. ]

For example, with a taker fee of 0.06 % (0.0006) and 10× leverage, the fees consume 1.2 % of the margin. To ensure the trade truly becomes risk-free when the stop moves, the break-even trigger should exceed this fee-adjusted ROI plus a small buffer. In the future version the bot should set

[ = (F_{} + F_{}) L + . ]

This modification ensures that the stop covers trading costs and protects against stop-loss hunts.

## 7 Liquidation price and maintenance margin

Leverage brings the risk of forced liquidation if the account equity falls below the maintenance margin. Bitget's guide provides a general formula for the liquidation price of a futures position 【420064011491786†L136-L153】:

[ = P_0 (1 + M), ]

where $M$ is the maintenance margin percentage specified by the exchange, and the "+¿" sign applies to short positions while the "−" sign applies to longs. The maintenance margin accounts for the minimum collateral the exchange requires to keep the position open. For example, a long entry at \$10,000 with 10× leverage and a 0.5 % maintenance margin yields a liquidation price of

$$10,000 - \left(\frac{10,000}{10} \times (1+0.005)\right) = 8,995\,\text{USDT} \quad 【420064011491786†L168-L177】.$$

Higher leverage reduces the buffer between the entry and liquidation price, underscoring the importance of conservative stop-losses 【109569485076299†L205-L219】.

## 8 Slippage and execution price

Market orders used for stop-losses or take-profits may execute at a different price than the trigger price, particularly in thin or volatile markets. Equiti's educational article defines slippage as the percentage difference between the execution price and the expected price 【293258100693517†L144-L158】:

[ = () . ]

Negative slippage occurs when a long order executes at a higher price or a short order executes at a lower price than requested, increasing the break-even requirement 【293258100693517†L151-L158】. To mitigate slippage, bots can incorporate a **slippage buffer** into trailing stops by widening the stop price by a small percentage (e.g., 0.02 % of price) to account for expected execution differences. Trading during high-liquidity periods, avoiding high-impact news events, and using stop-limit orders (rather than pure market stops) also reduce slippage risk 【293258100693517†L161-L186】.

## 9 API error handling and rate limits

Exchanges impose rate limits on API calls. A "cancel-then-replace" stop-update sequence exposes the position to market risk during the brief window between cancelling the old stop and placing the new one. To mitigate this:

1. **Batch operations**: when possible, use a single call to adjust a stop rather than separate cancel and replace calls. If the API does not support this, space requests to avoid hitting the rate limit and implement an exponential back-off on HTTP 429 errors.
2. **Retry queue**: when a stop placement fails due to network issues or rate limits, enqueue the request and retry after a delay. Persist the queue to disk so that failed operations are not lost if the process crashes.
3. **Reduce-only orders**: mark all stop and take-profit orders with the `reduceOnly` flag so they can only decrease or close an existing position. Bitunix notes that a reduce-only order "ensures that any new order you place will only reduce your existing position rather than increase it" 【294957825375978†L72-L81】. This prevents accidental reversal of a position if the bot or user closes the trade while a replacement stop is being submitted.
4. **Graceful degradation**: if the bot detects that a replacement stop failed, notify the user and consider immediately placing a market order to close the position rather than leaving it unprotected.

Since KuCoin's documentation is not publicly accessible from this environment, specific numeric rate limits are omitted. Traders should consult the exchange's API documentation for up-to-date limits and implement appropriate throttling.

## 10 Advanced trailing variants

While the bot's default staircase trailing stop suits most markets, alternative trailing methods can improve performance:

- **ATR-based trailing**: use the Average True Range to measure current volatility. Set the trailing distance as a multiple of ATR (e.g., 1.5 × ATR) rather than a fixed percentage. This adapts the stop to changing market conditions: wider during volatile periods and tighter during calm periods.
- **Time-based stops**: if a position does not achieve a defined profit within a set time (e.g., 30 minutes), move the stop to break-even or close the trade. This prevents capital from being tied up in stagnating positions.

- **Dynamic trailing step**: instead of a fixed 0.15 % ROI increment, scale the step size based on volatility or the magnitude of the open profit. For example, use smaller steps when ROI is below 5 % and larger steps when ROI exceeds 20 %.

These variants require more computation but may improve the risk/reward profile, especially in fast-changing crypto markets.

## 11 Scaling out (partial take-profits)

The current system closes the entire position at the take-profit or stop. Implementing partial exits can lock in gains while allowing the remainder to run. For example, close 50 % of the position at the first take-profit target and let the rest trail:

1. Define laddered targets, e.g., TP1 at 1 % ROI and TP2 at 3 % ROI.
2. Place stop-loss and take-profit orders for each portion of the position using reduceOnly orders so that the sum of the partial exits does not exceed the initial size.
3. After TP1 is hit, update the stop on the remaining half to break-even or higher to ensure no loss on the remainder.

Tracking the actual position size and adjusting subsequent stop and TP levels prevents inconsistent order sizes.

## 12 Security best practices

Interacting with exchange APIs carries operational and security risks. To reduce these risks:

- **API key permissions**: create separate API keys with only the permissions required for trading (typically “trade” and “read” but not “withdraw”).
- **IP whitelisting**: restrict API keys to known IP addresses so that the keys cannot be used from unauthorized locations.
- **Secure storage**: store API secrets in environment variables or encrypted vaults rather than in source code.
- **Two-factor authentication**: enable MFA on exchange accounts to prevent unauthorized access.
- **Reduce-only orders**: as described above, ensure that exit orders cannot accidentally reverse positions【294957825375978†L72-L81】.

By following these practices, traders minimise the risk of API key theft or unintended account actions.

## 13 Fee-adjusted break-even math (section 9)

In section 6 we introduced the general formula for net P&L. To incorporate fees directly into break-even logic, define the **fee-adjusted break-even trigger** as

[ = (F_{} + F_{}) L + , ]

where $F_{\text{entry}}$ and $F_{\text{exit}}$ are the maker/taker fee rates expressed as decimals (e.g., 0.0006 for 0.06 %), $L$ is the leverage, and buffer is a small safety margin (such as 0.1 %). The bot should

compare the leveraged ROI to this threshold rather than a fixed 0.001 %. For instance, with a 0.06 % fee and 10× leverage, the break-even ROI must exceed $(0.0006+0.0006)\times 10\times 100=1.2\%$ before the stop is moved to entry.

## 14 Trailing stop staircase algorithm (section 10)

The staircase algorithm described in section 5 produces discrete jumps in the stop-loss as profits accumulate. The logic can be summarised as follows:

1. **Trigger**: wait until the break-even flag is set.
2. **Measure profit**: compute the current leveraged ROI.
3. **Count steps**: determine how many TRAILING_STEP_PERCENT intervals have been crossed since the last trail:

$$\text{steps}=\left\lfloor \frac{\text{ROI}-\text{lastLevel}}{\text{TRAILING\_STEP\_PERCENT}} \right\rfloor .$$

4. **Move the stop**: if steps > 0, increase (long) or decrease (short) the stop price by steps × TRAILING_MOVE_PERCENT and record lastLevel = ROI.
5. **Repeat**: on each price update, re-evaluate steps. The stop never moves backward, creating a ratcheting effect.

This discrete approach reduces API calls compared with continuously updating the stop on every tick and provides a clear visual “staircase” of locked-in profits.

## 15 Auto-leverage via volatility (section 11)

Version 3.4.2’s dashboard includes an **Auto** mode for leverage selection based on the Average True Range (ATR). The idea is that leverage should decrease as volatility increases to reduce liquidation risk. Volatility is calculated as the ATR (over 14 periods) divided by the current price:

$$\text{Volatility}\%=\frac{\text{ATR}_{14}}{P}\times 100 .$$

The bot uses a tiered system to cap leverage:

| Volatility (ATR %) | Safe leverage cap |
|---|---|
| < 0.5 % | 50× |
| 0.5 %–1.0 % | 25× |
| 1.0 %–2.0 % | 15× |
| 2.0 %–3.0 % | 10× |
| > 5.0 % | 3× |

Users can adjust this baseline using a **risk multiplier** $M_{\text{risk}}$ to scale the suggested leverage, and the final leverage is clamped between 1× and 100×. For example,

$$\text{FinalLeverage} = min\left(100, max\left(1, round\left(\text{BaseLeverage} \times M_{\text{risk}}\right)\right)\right).$$

This volatility-aware leverage helps prevent over-exposure during turbulent markets.

## 16 API race conditions & safety (section 12)

When moving a stop-loss, the bot cancels the existing stop order and places a new one. There is a millisecond-scale window where the position is left without protection. In v3.4.2 this window is typically less than 500 ms, but unexpected API latency or failures could expose the position. To enhance safety:

1. **Use reduce-only flags** on all exit orders. Bitunix emphasises that reduce-only orders guarantee the new order can only close or decrease an existing position 【294957825375978†L72-L81】. This avoids the risk of inadvertently opening a short when closing a long.
2. **Retry on error**: wrap API calls in try/catch blocks and implement an automatic retry queue for failed stop placements. Persist this queue so that pending cancellations or replacements are not lost on crash.
3. **Rate limit awareness**: monitor the exchange's 429 or 429000 errors indicating too many requests, back off for several seconds, and resume after the window resets. Avoid sending bursts of requests when moving multiple stops simultaneously.
4. **User notification**: if a stop is cancelled but the replacement fails, immediately notify the user and consider executing a market order to close the position manually.
5. **Potential improvements**: future versions could update stops via WebSocket order amendments (if supported) or use OCO (one-cancels-the-other) orders to avoid explicit cancels.

These safeguards reduce the probability of being stopped out at a worse price or of having unprotected positions during API congestion.

## Conclusion

This manual extends the existing trailing stop and break-even guide into a comprehensive developer and trader's reference. It formalises the position-sizing and P&L calculations used in version 3.4, adds ROI-based stop logic, and introduces fee-aware break-even thresholds. It also provides formulas for liquidation price and slippage, proposes advanced trailing and scaling techniques, and outlines security and API-handling best practices. Incorporating these refinements into future versions of the bot will enhance risk management, reduce operational vulnerabilities and enable more adaptable trading strategies.